Anexo 3A

Webpack + TSX - Configuración de Bundling

Curso: TypeScript

60 min

Objetivos de Aprendizaje

- Comprender qué es un bundler y por qué se necesita
- Configurar webpack desde cero para proyecto TypeScript + React
- Integrar ts-loader y loaders necesarios (css, style, file)
- Migrar de ts-loader a esbuild-loader para mayor velocidad
- Configurar HtmlWebpackPlugin y DefinePlugin para variables de entorno
- Configurar tsconfig.json para JSX/React
- Implementar alias de importación para rutas limpias
- Implementar tree shaking y optimización de bundles
- Aplicar code splitting y lazy loading en React
- Diferenciar configuración dev vs production
- Configurar proxy para API calls en desarrollo
- Implementar Hot Module Replacement (HMR)

1. Contexto: ¿Por qué necesitamos un bundler?

10 minutos

El problema del desarrollo moderno

Sin bundler:

```
index.html
├── app.js        ← múltiples archivos
├── utils.js
├── components/
│   ├── Button.js
│   ├── Card.js
│   └── ...
└── styles/
    └── main.css
```

Problemas:

- Múltiples archivos HTTP → lentitud
- Dependencias no resueltas
- No hay minificación
- No hay tree shaking

¿Qué es Webpack?

Webpack es un **module bundler** que:

- Resuelve dependencias entre módulos
- Empaqueta todos los archivos en bundles
- Optimiza (minifica, tree shaking)
- Transpila (TS → JS, SCSS → CSS)
- Servidor de desarrollo con HMR

Conceptos clave: Entry, Output, Loaders, Plugins, Mode

2. Estructura básica de webpack

20 minutos

webpack.config.js básico

```
// webpack.config.js
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');

module.exports = {
  entry: './src/index.tsx',
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: '[name].[contenthash].js',
    publicPath: '/',  // CRÍTICO: URL base para assets en HTML
    clean: true,
  },
  cache: { type: 'filesystem' },  // Builds 10x más rápidos
  resolve: {
    extensions: ['.tsx', '.ts', '.js', '.jsx'],
    alias: {
      '@': path.resolve(__dirname, 'src'),
      '@components': path.resolve(__dirname, 'src/components'),
    }
```

Parámetros principales

- **entry**: Punto de entrada de la aplicación
- **output**: Configuración de archivos generados
- **publicPath**: *CRÍTICO* - URL base para assets en HTML (SPAs fallan sin esto)
- **cache**: Cache filesystem para builds 10x más rápidos
- **resolve**: Resolución de archivos y aliases
- **module.rules**: Reglas para transformar archivos
- **optimization**: moduleIds estables y runtime separado
- **plugins**: Funcionalidades adicionales
- **mode**: development / production

Documentación: Webpack Configuration

Lab: Configuración inicial del proyecto

```
# 1. Inicializar proyecto
npm init -y

# 2. Instalar dependencias
npm install react react-dom
npm install -D webpack webpack-cli webpack-dev-server typescript ts-loader

# 3. Crear estructura
mkdir -p src/components src/pages public
touch webpack.config.js
```

3. Configuración de TypeScript para React

15 minutos

tsconfig.json para React

```
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["DOM", "DOM.Iterable", "ES2020"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "isolatedModules": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Clave: `"jsx": "react-jsx"` transpila JSX sin importar React

Lab: Configurar tsconfig.json para React

```
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["DOM", "DOM.Iterable", "ES2020"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true
  },
  "include": ["src"],
  "exclude": ["node_modules"]
}
```

4. Loaders y plugins

15 minutos

¿Qué son los loaders?

Los loaders transforman archivos específicos antes de incluirlos en el bundle:

- **ts-loader**: Transpila TypeScript a JavaScript
- **css-loader + style-loader**: Manejan CSS
- **file-loader**: Imágenes y fuentes

```
// ts-loader
{ test: /\.tsx?$/, use: 'ts-loader', exclude: /node_modules/ }

// css-loader + style-loader
{ test: /\.css$/, use: ['style-loader', 'css-loader'] }
```

Lab: Añadir loaders y configurar plugins

```
// webpack.config.js
module.exports = {
  module: {
    rules: [
      {
        test: /\.tsx?$/,
        use: 'ts-loader',
        exclude: /node_modules/,
      },
      {
        test: /\.css$/,
        use: ['style-loader', 'css-loader'],
      },
    ],
  },
  plugins: [
    new HtmlWebpackPlugin({
      template: './public/index.html',
    })
```

5. Transpilación avanzada: esbuild-loader

15 minutos

¿Por qué alternativas a ts-loader?

- **esbuild-loader**: 10-100x más rápido que ts-loader
- **babel-loader**: Flexibilidad con presets
- **swc-loader**: Rápido, escrito en Rust

¿Por qué esbuild-loader? Velocidad extrema (esbuild está escrito en Go), soporte nativo para TSX

Migrar de ts-loader a esbuild-loader

```
// Reemplazar ts-loader por esbuild-loader
{
  test: /\.tsx?$/,
  loader: 'esbuild-loader',
  options: {
    loader: 'tsx',
    target: 'es2020',
  },
}
```

Lab: Migrar de ts-loader a esbuild-loader

```
# Instalar esbuild-loader
npm install --save-dev esbuild-loader
```

```
// webpack.config.js - actualizar rule
module.exports = {
  module: {
    rules: [
      {
        test: /\.tsx?$/,
        loader: 'esbuild-loader',
        options: {
          loader: 'tsx',
          target: 'es2020',
        },
      },
    ],
  },
};
```

6. Tree shaking y optimización

15 minutos

¿Qué es tree shaking?

Eliminación de código muerto durante el bundling:

- Elimina funciones/exportaciones no utilizadas
- Reduce tamaño del bundle final
- Funciona con ES6+ imports/exports estáticos

```
// webpack eliminará 'resta' del bundle
import { suma } from './utils/math';
```

Configuración de sideEffects

```
{
  "name": "tareas-app",
  "sideEffects": ["*.css", "*.scss"]
}
```

- `false` = todo el código es "seguro" de eliminar
- Array = lista de archivos con side effects (CSS)

Lab: Configurar tree shaking

```
// src/utils/math.ts
export const suma = (a: number, b: number) => a + b;
export const resta = (a: number, b: number) => a - b;
export const multiplica = (a: number, b: number) => a * b;

// src/App.tsx - solo importamos suma
import { suma } from './utils/math';

// En production, webpack elimina 'resta' y 'multiplica'
```

7. Code Splitting y Lazy Loading

15 minutos

¿Qué es code splitting?

Divide el bundle en chunks separados:

- Carga bajo demanda solo el código necesario
- Reduce tiempo de carga inicial
- Esencial para aplicaciones React grandes

```
import { lazy, Suspense } from 'react';

const TaskList = lazy(() => import('./components/TaskList'));

function App() {
  return (
    <Suspense fallback={<div>Cargando...</div>>
      <TaskList />
    </Suspense>
  );
}
```

SplitChunksPlugin

```
optimization: {
  splitChunks: {
    chunks: 'all',
    cacheGroups: {
      vendor: { test: /[\\/]node_modules[\\/]/, name: 'vendors', chunks: 'all' },
    },
  },
}
```

Lab: Implementar lazy loading en React

```
// src/App.tsx
import { lazy, Suspense } from 'react';

const TaskList = lazy(() => import('./components/TaskList'));
const UserPanel = lazy(() => import('./components/UserPanel'));
const TaskForm = lazy(() => import('./components/TaskForm'));

function App() {
  return (
    <Suspense fallback={<div>Cargando...</div>}>
      <TaskList />
      <UserPanel />
      <TaskForm />
    </Suspense>
  );
}

// webpack.config.js - splitChunks
optimization: {
```

8. Modes y scripts

15 minutos

Modes: development vs production

Mode	Propósito	Características
development	Desarrollo rápido	Sin minificación, source maps completos
production	Deploy	Minificación, tree shaking, optimización

Scripts en package.json

```
{
  "scripts": {
    "start": "webpack serve --mode development",
    "build": "webpack --mode production",
    "build:dev": "webpack --mode development"
  }
}
```

Variables de entorno con DefinePlugin

```
new webpack.DefinePlugin({
  'process.env.API_URL': JSON.stringify(process.env.API_URL || 'http://localhost:3001'),
  'process.env.NODE_ENV': JSON.stringify(process.env.NODE_ENV || 'development'),
})
```

Lab: Configurar scripts y variables de entorno

```
// package.json
{
  "scripts": {
    "dev": "webpack serve --mode development",
    "build": "webpack --mode production",
    "start": "webpack serve --mode development --open",
    "typecheck": "tsc --noEmit"
  }
}

// webpack.config.js
module.exports = {
  plugins: [
    new webpack.DefinePlugin({
      'process.env.API_URL': JSON.stringify(process.env.API_URL || 'http://localhost:3001'),
      'process.env.VERSION': JSON.stringify('1.0.0'),
    }),
  ],
};
```

9. Source Maps avanzados

10 minutos

Tipos de Source Maps

Tipo	Uso	Velocidad
eval	Desarrollo rápido	Muy rápida
cheap-module-eval-source-map	Desarrollo	Rápida
inline-source-map	Debugging	Lenta
source-map	Producción	Muy lenta

Lab: Configurar source maps para debugging

```
// webpack.config.js
module.exports = (env, argv) => ({
  // Desarrollo: rápido y legible
  devtool: 'cheap-module-eval-source-map',

  // Producción: source-map para debugging
  ...(argv.mode === 'production' && {
    devtool: 'source-map',
  }),

  // Configuración completa
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: '[name].[contenthash].js',
  },
});
```

10. Bundle analysis

10 minutos

webpack-bundle-analyzer

¿Por qué analizar el bundle?

- Identificar dependencias grandes
- Detectar código duplicado
- Optimizar el tamaño del bundle

```
npm install --save-dev webpack-bundle-analyzer
```

Lab: Analizar bundle del proyecto React

```
// webpack.config.js
const BundleAnalyzerPlugin = require('webpack-bundle-analyzer').BundleAnalyzerPlugin;

module.exports = {
  plugins: [
    new BundleAnalyzerPlugin({
      analyzerMode: 'static',
      openAnalyzer: true,
      reportFilename: 'bundle-report.html',
    }),
  ],
};

// package.json
{
  "scripts": {
    "analyze": "webpack --profile --json > stats.json && webpack-bundle-analyzer stats.json"
  }
}
```

11. Errores comunes de configuración

10 minutos

Errores frecuentes y soluciones

Error	Causa	Solución
Module not found	Extensión no resuelta	Añadir extensiones en resolve.extensions
Cannot find module	Path incorrecto	Verificar alias y paths relativos
CSS not applied	Orden de loaders incorrecto	style-loader debe ir primero
HMR not working	Configuración HMR incorrecta	Verificar hot: true en devServer

Lab: Diagnosticar y resolver errores comunes

```
// Error 1: Module not found - resolver.extensions
resolve: {
  extensions: ['.tsx', '.ts', '.js', '.jsx'],  // ✅ Falta .tsx
}

// Error 2: Cannot find module - alias incorrecto
alias: {
  '@': path.resolve(__dirname, 'src'),  // ✅ Verificar path
}

// Error 3: CSS not applied - orden de loaders
module: {
  rules: [
    {
      test: /\.css$/,
      use: ['style-loader', 'css-loader'],  // ✅ style-loader primero
    },
  ],
}
```

12. HMR y DevServer

15 minutos

Hot Module Replacement (HMR)

- Actualiza módulos sin recargar toda la página
- Mantiene el estado de la aplicación
- Desarrollo más rápido

```
devServer: {
  port: 3000,
  hot: true,
  historyApiFallback: true,
  proxy: {
    '/api': { target: 'http://localhost:3001', changeOrigin: true }
  }
}
```

Lab: Configurar HMR y servidor de desarrollo

```
// webpack.config.js completo
module.exports = {
  mode: 'development',
  devServer: {
    port: 3000,
    hot: true,
    historyApiFallback: true,  // React Router
    open: true,
    proxy: [
      {
        context: ['/api', '/auth'],
        target: 'http://localhost:3001',
        changeOrigin: true,
      },
    ],
  },
  plugins: [
    new webpack.HotModuleReplacementPlugin(),
  ]
```

13. Resumen

5 minutos

Puntos Clave

1. Webpack es un bundler que empaqueta todos los recursos
2. ts-loader transpila TypeScript a JavaScript
3. esbuild-loader es 10-100x más rápido que ts-loader
4. Tree shaking elimina código muerto en production
5. Code splitting divide el bundle en chunks separados
6. React.lazy() permite carga bajo demanda de componentes
7. tsconfig.json debe tener `jsx: "react-jsx"` para React
8. css-loader + style-loader manejan CSS
9. HtmlWebpackPlugin genera el HTML con el bundle incluido
10. DefinePlugin permite usar variables de entorno en el código
11. Alias facilitan las importaciones (`@/components`)
12. Proxy del devServer redirige API calls durante desarrollo
13. Mode determina el nivel de optimización (dev/production)
14. HMR permite desarrollo rápido sin perder estado
15. webpack-bundle-analyzer ayuda a optimizar el bundle

14. Ejercicio Integrador

Configurar Caso 2r (Sistema de Biblioteca) con Webpack

Requisitos

- Configurar webpack para el proyecto de biblioteca
- Soporte para componentes .tsx
- Migrar a esbuild-loader
- Implementar tree shaking
- Aplicar code splitting con React.lazy()
- Soporte para estilos .css
- Servidor de desarrollo con HMR
- Build de producción optimizado
- Análisis de bundle con webpack-bundle-analyzer

15. Referencias

Recursos Adicionales

- Webpack Documentation
- ts-loader
- esbuild-loader
- webpack-bundle-analyzer
- HtmlWebpackPlugin
- Webpack Dev Server
- TypeScript + Webpack Guide
- React.lazy

Gracias!

Anexo 3A Completado

Siguiente: Anexo 4A - Tipos en React